Chapitre 06 — Traiter du texte avec sed et awk

Objectif Transformer un fichier texte, sélectionner des colonnes et calculer un total, sans modifier l'original par erreur.

1. Choisir l'outil selon la tâche

Besoin	Outil
Garder des lignes contenant un mot	`grep`
Remplacer un mot ou sélectionner des lignes	`sed`
Lire des champs et calculer	`awk`

Les trois outils lisent le texte. **Par défaut, ils ne changent pas le fichier lu.** Ils envoient un résultat sur leur sortie standard.

2. Préparer un petit inventaire

Dans Bash sur Debian, colle le bloc complet jusqu'à `EOF` :

```
mkdir -p ~/atelier-bash/texte
cd ~/atelier-bash/texte
cat > stock.csv <<'EOF'
produit;categorie;quantite;prix
clavier;informatique;8;25
souris;informatique;12;15
cahier;papeterie;20;3
ecran;informatique;4;180
stylo;papeterie;30;2
EOF
```

`<<'EOF'` fournit les lignes suivantes à `cat`, sans interpréter leur contenu. `>` écrit le résultat dans `stock.csv`. La dernière ligne `EOF` doit être seule.

Cette écriture remplace avantageusement les `echo` du chapitre 05 dès qu'il y a plus de deux ou trois lignes à écrire : le contenu du fichier apparaît tel quel dans le script, sans guillemets ni `\n` à gérer.

Ce fichier utilise `;` pour séparer quatre champs. La première ligne est un en-tête. C'est un format simple : aucun champ ne contient de point-virgule ni de saut de ligne. Le `awk` de ce cours ne remplace pas un lecteur CSV capable de traiter tous les cas de champs entre guillemets.



Une ligne découpée en quatre champs

3. Remplacer du texte avec sed

```
sed 's/informatique/materiel/g' stock.csv
```

Lis l'expression de gauche à droite :

Partie	Rôle
`s`	Substitution
`informatique`	Texte recherché dans cet exemple
`materiel`	Texte de remplacement
`g`	Toutes les occurrences sur chaque ligne

Les guillemets simples protègent l'expression du shell. La recherche de `sed` utilise des expressions régulières ; ici, nous cherchons un mot sans caractère spécial.

À quoi sert vraiment le `g` ? Sur l'exemple ci-dessus, il ne sert à rien : aucune ligne ne contient deux fois `informatique`, donc avec ou sans `g` le résultat est identique. Son effet n'apparaît que lorsqu'un motif revient plusieurs fois sur la même ligne — le point-virgule, par exemple :

```
echo 'stylo;papeterie;30;2' | sed 's/;/ | /'
echo 'stylo;papeterie;30;2' | sed 's/;/ | /g'
```

```
stylo | papeterie;30;2
stylo | papeterie | 30 | 2
```

Sans `g`, `sed` ne remplace que **la première** occurrence de chaque ligne. C'est l'oubli le plus fréquent avec `s///`.

Pour conserver le résultat :

```
sed 's/informatique/materiel/g' stock.csv > stock-corrige.csv
cat stock-corrige.csv
cat stock.csv
```

Le nouveau fichier contient `materiel`. L'original contient toujours `informatique`.

Deux noms de fichiers différents N'écris jamais `sed ... stock.csv > stock.csv`. Le shell viderait le fichier de destination avant que `sed` puisse le lire.

4. Choisir ou retirer des lignes

```
sed -n '2,4p' stock.csv
```

`-n` coupe l'affichage automatique ; `2,4p` affiche les lignes 2 à 4 incluses. Tu obtiens clavier, souris et cahier.

```
sed '1d' stock.csv
```

`1d` retire la première ligne de la **sortie**. Le fichier n'est pas modifié.

```
sed '/papeterie/d' stock.csv
```

Ce dernier exemple retire de la sortie les lignes contenant `papeterie`.

5. Lire les champs avec awk

Un programme `awk` s'écrit toujours sur le même modèle :

```
awk -F 'separateur' 'condition { action }' fichier
```

La **condition** choisit les lignes, l'**action** dit quoi en faire. Appliqué à notre inventaire :

```
awk -F ';' 'NR > 1 { print $1, $3 }' stock.csv
```

- `-F ';'` fixe le séparateur de champs.
- `NR` est le numéro de la ligne en cours.
- `NR > 1` saute l'en-tête : c'est la condition.
- `{ print $1, $3 }` est l'action : `$1` est le produit, `$3` sa quantité.

Résultat :

```
clavier 8
souris 12
cahier 20
ecran 4
stylo 30
```

Les `$1` et `$3` appartiennent ici à **awk**, pas à Bash : les guillemets simples empêchent Bash de les remplacer.

Les deux morceaux sont facultatifs C'est ce qui rend les programmes `awk` si variés d'aspect :

Programme	Sens
`'NR > 1 { print $1 }'`	Condition **et** action
`'{ print $1 }'`	Pas de condition → l'action s'applique à **toutes** les lignes
`'NR > 1'`	Pas d'action → awk affiche la ligne entière

Garde ce tableau en tête : tu croiseras les trois formes, y compris au §8 de ce chapitre.

6. Filtrer avec une condition

```
awk -F ';' 'NR > 1 && $3 < 10 { print $1, $3 }' stock.csv
```

La ligne doit être après l'en-tête **et** sa quantité doit être inférieure à 10. Résultat : `clavier 8`, puis `ecran 4`.

> Un champ n'est pas forcément un nombre `awk` compare numériquement quand les deux côtés ressemblent à des nombres. Sinon, il compare comme du texte — sans prévenir :

```
champ [8]     → selectionne     (8 < 10)
champ [80]    → ecarte          (80 > 10)
champ [huit]  → ecarte          compare comme du texte
champ []      → SELECTIONNE     un champ vide compte pour 0
```

> Le dernier cas est le piège : une quantité manquante est signalée comme un stock faible. Avant de filtrer un fichier qui n'est pas le tien, regarde-le avec `sed -n '1,5p'`.

7. Calculer un total

```
awk -F ';' 'NR > 1 { total += $3 } END { print total }' stock.csv
```

À chaque produit, `total += $3` ajoute sa quantité. Après la dernière ligne, `END` affiche la somme. Résultat : **74 articles**.

Tu remarques qu'on n'initialise pas `total`, contrairement au compteur du chapitre 05. C'est une particularité d'`awk` : une variable jamais définie vaut la chaîne vide, et `0` dès qu'on s'en sert comme nombre. En Bash, il aurait fallu écrire `total=0` avant la boucle.

Pour calculer la valeur de l'inventaire :

```
awk -F ';' 'NR > 1 { total += $3 * $4 } END { print total }' stock.csv
```

Chaque quantité est multipliée par son prix unitaire. Résultat : **1220**.

Le `END` d'`awk` n'est pas le `done` d'une boucle Bash : nous utilisons deux langages différents, chacun avec sa syntaxe.

8. Relier deux outils

```
sed '1d' stock.csv | awk -F ';' '{ total += $3 } END { print total }'
```

`sed` retire l'en-tête, puis le tube transmet les produits à `awk`. On retrouve 74.

Remarque le programme `awk` : il n'a plus de condition devant `{ total += $3 }`, puisque l'en-tête a déjà été écarté par `sed`. C'est la deuxième forme du tableau du §5 — l'action s'applique à toutes les lignes reçues.

C'est une autre façon de faire ; le `awk` du §7 suffit quand il sait filtrer lui-même l'en-tête.

9. Modifier un fichier avec une sauvegarde

Jusqu'ici, aucune commande n'a touché à `stock.csv`. Pour modifier un fichier sur place, `sed` a l'option `-i` — à n'utiliser qu'avec une sauvegarde, et d'abord sur une copie :

```
cp stock.csv stock-test.csv
sed -i.bak 's/informatique/materiel/g' stock-test.csv
cat stock-test.csv
cat stock-test.csv.bak
```

Sur Debian, `-i.bak` modifie `stock-test.csv` et conserve l'ancienne version dans `stock-test.csv.bak`. Ce comportement est celui de GNU sed ; ne suppose pas que les options sont identiques sur tous les systèmes.

Le défi supplémentaire du TP demande cette option : ce n'est pas une notion facultative.

À retenir

- `sed` transforme ou sélectionne du texte ; `awk` travaille avec des champs.
- `cat > fichier <<'EOF'` écrit plusieurs lignes d'un coup, telles quelles.
- `s/recherche/remplacement/g` : sans le `g`, seule la **première** occurrence de chaque ligne est remplacée.
- `sed -n '2,4p'` garde des lignes, `sed '1d'` et `sed '/motif/d'` en retirent — toujours sur la sortie, jamais sur le fichier.
- Un programme `awk` s'écrit `'condition { action }'`. Sans condition, l'action vaut pour toutes les lignes ; sans action, la ligne entière est affichée.
- `-F ';'` fixe le séparateur, `NR` donne le numéro de ligne, `NR > 1` saute l'en-tête, `END` s'exécute après la dernière ligne.
- Une variable `awk` n'a pas besoin d'être initialisée : elle vaut `0` dès le premier calcul.
- Identifie le séparateur, l'en-tête et les champs vides avant de calculer.
- Regarde d'abord le résultat, puis écris dans un nouveau fichier. `sed -i.bak` seulement en connaissance de cause.

À toi : que désigne `$4` dans notre inventaire ? Pourquoi utilisons-nous `NR > 1` ? Pourquoi des guillemets simples autour du programme `awk` ? Que ferait `sed 's/;/,/'` sans le `g` sur une ligne de ce fichier ?

Passer au TP